Национальный исследовательский университет "Высшая школа экономики"
Факультет компьютерных наук
Образовательная программа "Программная инженерия"

СОГЛАСОВАНО	УТВЕРЖДЕНО
Старший преподаватель: Департамент анализа данных и искусственного интеллекта, НИУ ВШЭ	Академический руководитель образовательной программы "Программная инженерия", старший преподаватель департамента программной инженерии
_____ А. А. Паринов	_____ Н. А. Павлочев
«___» _____ 2026 г.	«___» _____ 2026 г.

ЦИФРОВОЙ АССИСТЕНТ СТУДЕНТА

Программа и методика испытаний

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.12.17 51 01-1-ЛУ

Исполнители:

Студент группы БПИ236
_____ / С. Е. Растворов /
«___» _____ 2026 г.

Студент группы БПИ236
_____ / Н. Д. Радионов /
«___» _____ 2026 г.

2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подп	

УТВЕРЖДЕН

RU.17701729.12.17 51 01-1-ЛУ

ЦИФРОВОЙ АССИСТЕНТ СТУДЕНТА

Программа и методика испытаний

RU.17701729.12.17 51 01-1

Листов 17

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подп	

Исполнители:

Студент группы БПИ236

_____ / С. Е. Растворов /

«___» _____ 2026 г.

Студент группы БПИ236

_____ / Н. Д. Радионов /

«___» _____ 2026 г.

2026

АННОТАЦИЯ

Программа и методика испытаний определяет объект, цель, средства, порядок и методы испытаний программного продукта «Цифровой Ассистент Студента».

Документ предназначен для проверки соответствия программы требованиям технического задания и фиксации критериев приемки результата разработки.

СОДЕРЖАНИЕ

1. ОБЪЕКТ ИСПЫТАНИЙ . . . 4
1.1. Наименование программы . . . 4
1.2. Краткая характеристика области применения программы . . . 4
2. ЦЕЛЬ ИСПЫТАНИЙ . . . 5
3. ТРЕБОВАНИЯ К ПРОГРАММЕ . . . 6
3.1. Требования к функциональным характеристикам . . . 6
3.1.1. Состав выполняемых функций . . . 6
3.1.2. Требования к входным и выходным данным . . . 7
3.1.3. Организация входных данных . . . 7
3.1.4. Организация выходных данных . . . 8
3.1.5. Требования к интерфейсу . . . 8
4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ . . . 10
5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ . . . 11
5.1. Технические средства . . . 11
5.2. Программные средства . . . 11
5.3. Порядок проведения испытаний . . . 11
5.4. Требования к персоналу . . . 12
6. МЕТОДЫ ИСПЫТАНИЙ . . . 13
6.1. Проверка требований к технической документации . . . 13
6.2. Проверка требований к интерфейсу . . . 13
6.3. Проверка требований к функциональным характеристикам . . . 13
6.3.1. Основной интеграционный сценарий . . . 14
6.3.2. Проверка создания, редактирования и модерации проекта . . . 14
6.3.3. Проверка обработки заявок студентов . . . 14
6.3.4. Проверка каталога, поиска и рекомендаций . . . 14
6.3.5. Проверка импорта и актуализации данных . . . 15
6.3.6. Проверка домена факультета и сопоставления научных руководителей . . . 15
6.3.7. Проверка агрегированных показателей и контуров синхронизации . . . 15
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ . . . 16

1. ОБЪЕКТ ИСПЫТАНИЙ

1.1. Наименование программы

Наименование программы: «Цифровой Ассистент Студента: серверная часть».

Наименование программы на английском языке: «Digital Student Assistant».

1.2. Краткая характеристика области применения программы

Веб-платформа для подбора и сопровождения студенческих проектов ВШЭ. Система объединяет каталог проектов, импорт ЭПП из XLSX, роли студентов, заказчиков и сотрудников ЦППРП, модерацию, заявки, избранное, интересы и рекомендации. Основной контур реализован как Django + DRF с HTML-интерфейсом на Django templates, отдельным FastAPI ML-сервисом и graph-сервисом на Neo4j. Серверная часть публикует доменные события через outbox API доставки событий, а внешние потребители используют их для обновления индекса рекомендаций и графовой проекции. В актуальной версии добавлены подтверждение адреса электронной почты при регистрации, django-unfold admin, домен faculty для синхронизации преподавателей/публикаций/курсов и сопоставления научных руководителей проектов с профилями HSE. Целевое окружение использует PostgreSQL, Docker Compose, Nginx и сценарии для развертывания, резервного копирования и синхронизации окружения.

2. ЦЕЛЬ ИСПЫТАНИЙ

Цель испытаний состоит в проверке корректности функционирования программного комплекса «`project-name`» (`#project-name-english`) по требованиям технического задания: веб-интерфейса на Django templates, серверной части на Django + DRF, сервиса рекомендаций и поиска, сервиса графовой проекции, а также контуров импорта, экспорта и фоновой синхронизации данных.

В ходе испытаний должна быть подтверждена работоспособность основных пользовательских сценариев для студента, заказчика и сотрудника ЦППРП, включая аутентификацию, подтверждение электронной почты, просмотр каталога проектов, работу с карточкой проекта, подачу и обработку заявок, управление избранным и интересами, модерацию проектов, импорт проектов из XLSX, формирование рекомендаций и передачу событий во внутренние интеграционные контуры.

Границы приемки определяются проверкой только тех функций, которые реализованы в текущей версии системы и доступны через веб-интерфейс, REST API и служебные интеграционные интерфейсы проекта. Испытания не должны требовать подтверждения заявленных в исходном ТЗ возможностей, которые отсутствуют в текущем коде или прямо отмечены как не подтвержденные фактической реализацией.

Результаты испытаний считаются положительными, если все проверяемые сценарии выполняются без критических ошибок, ответы API и действия интерфейса соответствуют ожидаемому поведению, ограничения прав доступа соблюдаются, импорт и обработка данных выполняются без нарушения целостности, а интеграционные связи между веб-приложением, сервисом рекомендаций, графовым сервисом и контуром outbox работают согласованно.

3. ТРЕБОВАНИЯ К ПРОГРАММЕ

3.1. Требования к функциональным характеристикам

Программа должна соответствовать следующим функциональным требованиям, указанным в документе «Цифровой Ассистент Студента: серверная часть. Техническое задание», и проверяться по ним с учетом заявленного состава интеграций проекта.

3.1.1. Состав выполняемых функций

Программа Цифровой Ассистент Студента: серверная часть должна обеспечивать работу веб-платформы подбора и сопровождения студенческих проектов как целостного набора взаимосвязанных функций клиентской части, серверной части и фоновых сервисов. Функции должны быть доступны в зависимости от роли пользователя и состояния данных.

1. Клиентская часть должна обеспечивать:
- отображение каталога проектов, карточек проекта и сведений о статусах;
- фильтрацию, поиск и переход к детальной странице проекта;
- просмотр и редактирование собственных данных профиля, интересов и избранного;
- подачу заявки на проект, отзыв ранее поданной заявки, просмотр статусов и сообщений системы;
- создание и редактирование карточек проектов для пользователей с ролью заказчика;
- просмотр очереди модерации, обработку проектов и заявок для пользователей с ролью ЦППРП;
- работу с формами аутентификации, подтверждения email и восстановления действий после ошибок валидации;
- доступ к административному интерфейсу `django-unfold` для служебных операций, если пользователю назначены соответствующие права.
2. Серверная часть должна обеспечивать:
- регистрацию и аутентификацию пользователей по локальной учетной записи с подтверждением адреса электронной почты;
- выдачу и проверку токенов/сессионных данных для доступа к API и страницам кабинетов;
- хранение и обработку данных о пользователях, ролях, проектах, заявках, интересах и избранном;
- реализацию жизненного цикла проектов, включая черновик, модерацию, публикацию и снятие с публикации;
- обработку заявок студентов, фиксацию решений заказчика и модератора, хранение причин отклонения и комментариев;
- импорт проектов и связанных справочников из XLSX-файлов и командной строки;
- выгрузку и предоставление данных через API для кабинетов, внутренних страниц и административных сценариев;
- ведение outbox-потока доменных событий и выдачу данных для внешних обработчиков;
- поддержку домена `faculty`, включая синхронизацию преподавателей, публикаций, курсов и сопоставление научных руководителей с проектами;

- работу сервиса рекомендаций и поиска через фасад над ML-сервисом с резервным ранжированием по ключевым словам при отсутствии семантического ответа;
- поддержку графовой проекции через отдельный сервис и хранилище Neo4j;
- ведение журналов, метрик и служебных статусов для эксплуатационного контура.

3. Интеграции и фоновые задачи должны обеспечивать:

- обмен данными между веб-приложением, ML-сервисом и графовым сервисом по HTTP;
- обработку доменных событий `project.changed`, `application.changed`, `user_profile.changed`, `deadline.changed`, `import.completed`, `recs.reindex_requested`, `faculty.*` и связанных с ними событий;
- обновление поискового индекса и рекомендательной проекции после импорта и изменения данных;
- синхронизацию данных о faculty-сущностях и проектах-научруководителях;
- отправку уведомлений, связанных с подтверждением email и служебными действиями системы;
- эксплуатационные сценарии развертывания, резервного копирования, восстановления и синхронизации окружения.

3.1.2. Требования к входным и выходным данным

3.1.3. Организация входных данных

Входными данными для клиентской части являются пользовательские действия, данные форм и сведения, получаемые от сервера для отображения. Входными данными для серверной части являются HTTP-запросы, файлы импорта и внутренние сообщения интеграционных сервисов. Все входные данные должны проходить проверку структуры, обязательности полей и прав доступа до выполнения бизнес-операций.

1. Входные данные клиентской части включают:

- учетные данные пользователя, поля подтверждения email и данные аутентификации;
- параметры поиска, фильтрации и сортировки каталога проектов;
- содержимое форм создания и редактирования проекта;
- параметры заявки, решения по заявке, комментарии модерации и причины отклонения;
- данные профиля, интересов, избранного и настроек пользователя;
- подтверждения значимых действий, выполняемых через интерфейс.

2. Входные данные, поступающие от серверной части в клиентскую, включают:

- списки проектов, карточки проекта и агрегированные сведения о статусах;
- сведения о поданных заявках, результатах модерации и истории изменений;
- данные профиля пользователя, его ролей, интересов и избранного;
- уведомления, сообщения об ошибках и данные о валидации форм;
- данные для отображения результатов поиска, рекомендаций и связанных сущностей факультетского домена.

3. Входные данные серверной части включают:

- запросы браузера и клиентских форм в формате HTTP с полезной нагрузкой JSON или multipart/form-data;
- запросы к API `/api/v1/...` с параметрами фильтрации, идентификаторами сущностей и токенами доступа;
- XLSX-файлы импорта проектов и справочников, а также данные из командных и административных сценариев загрузки;
- события из outbox API доставки событий для обновления ML- и graph-проекций;
- запросы к сервису рекомендаций и поиска, включая HTTP-запросы на reindex и получение результатов поиска.

3.1.4. Организация выходных данных

Выходные данные программы должны обеспечивать корректное отображение результата работы пользователя в браузере и предоставление машинно-обрабатываемых ответов для внешних и внутренних потребителей. Формат и состав выходных данных должны соответствовать типу запроса и роли пользователя.

1. Выходные данные клиентской части включают:
- визуальные страницы, таблицы, карточки и формы веб-интерфейса;
- сообщения об успешном выполнении действий;
- сообщения об ошибках ввода и ограничениях доступа;
- состояния кнопок, форм и элементов навигации, отражающие результат серверной проверки;
- отображение email-верификации, статусов заявок, модерации и рекомендаций.
2. Выходные данные серверной части включают:
- ответы API в формате JSON с кодами HTTP 2xx/4xx/5xx и детализированными сообщениями об ошибках;
- данные для HTML-страниц и частичных обновлений интерфейса;
- результаты импорта, модерации, публикации и изменения статусов сущностей;
- экспортные файлы XLSX и CSV для административных сценариев и выгрузок;
- события outbox и данные для обновления поискового, рекомендательного и графового контуров;
- журналы операций, структурированные записи об ошибках и метрики эксплуатационного контроля.

3.1.5. Требования к интерфейсу

Клиентская часть системы должна предоставлять веб-интерфейс, доступный через современный браузер и построенный на ролевой модели доступа. Интерфейс должен поддерживать единый стиль навигации, понятную структуру разделов и валидацию вводимых данных до отправки на сервер и после получения ответа.

1. Общие требования к интерфейсу:
- интерфейс должен быть реализован как веб-приложение на Django templates;
- доступ к разделам должен ограничиваться правами текущей роли;
- формы должны сопровождаться сообщениями о некорректном вводе;

- значимые действия должны подтверждаться пользователем перед отправкой;
- результат операции должен быть виден без перезагрузки всей страницы, если это допускает текущая реализация страницы.

2. Интерфейс студента должен обеспечивать:
- просмотр каталога проектов;
- переход к карточке проекта;
- работу с поиском, фильтрами, избранным и интересами;
- подачу и отзыв заявки;
- просмотр статусов заявок и уведомлений;
- редактирование профиля и данных подтвержденного email.

3. Интерфейс заказчика должен обеспечивать:
- создание и редактирование проектов;
- управление стадиями жизненного цикла проекта;
- отправку проекта на модерацию;
- просмотр заявок по своим проектам;
- принятие решений по заявкам и управление квотами.

4. Интерфейс ЦППРП и администратора должен обеспечивать:
- просмотр очереди модерации;
- возврат проекта на доработку и публикацию;
- работу с агрегированными показателями и служебными данными;
- управление справочной информацией, дедлайнами и параметрами импорта;
- доступ к расширенному административному интерфейсу django-unfold.

5. Интерфейс взаимодействия с внешними сервисами должен обеспечивать:
- запуск и контроль импорта XLSX;
- просмотр результатов обработки событий outbox;
- доступ к данным рекомендаций и поиска;
- сопровождение синхронизации faculty-домена и проектных связей.

6. Основным режимом входа является локальная аутентификация с подтверждением адреса электронной почты; дополнительные способы входа могут проверяться только при их отдельном включении в утвержденный состав требований.

4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

На испытание должна быть представлена документация в следующем составе:

1. «Цифровой Ассистент Студента: серверная часть». Техническое задание (ГОСТ 19.201-78 [7]).
2. «Цифровой Ассистент Студента: серверная часть». Пояснительная записка (ГОСТ 19.404-79 [10]).
3. «Цифровой Ассистент Студента: серверная часть». Программа и методика испытаний (ГОСТ 19.301-79 [8]).
4. «Цифровой Ассистент Студента: серверная часть». Текст программы (ГОСТ 19.401-78 [9]).
5. «Цифровой Ассистент Студента: серверная часть». Руководство оператора (ГОСТ 19.505-79 [11]).

5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ

5.1. Технические средства

Для испытаний используется командный стенд, способный поднять полный контур проекта в контейнерах: веб-приложение `src/web`, ML-сервис `src/ml`, graph-сервис `src/graph`, PostgreSQL, Nginx и вспомогательные сервисы, определенные в `infra/docker-compose*.yml`.

На рабочем месте испытателя должен быть установлен современный веб-браузер для проверки пользовательских сценариев в интерфейсе на Django templates. Допускается использование актуальных версий Chrome, Firefox, Safari и Edge. При необходимости конкретные версии браузеров указываются в финальной редакции документа в соответствии с требованиями кафедры или площадки защиты.

Для проверки сценариев регистрации и подтверждения адреса электронной почты требуется доступ к SMTP-серверу или консольному почтовому механизму, используемому конфигурацией окружения. При проверке графовой проекции дополнительно поднимается Neo4j, поскольку без него невозможно подтвердить взаимодействие графового контура.

5.2. Программные средства

Испытания выполняются на стеке, соответствующем техническому заданию и реализации проекта: Python 3.13+, Django 5+ и Django REST Framework для основного серверного контура, FastAPI для ML- и graph-сервисов, PostgreSQL для основного хранилища данных, Docker и Docker Compose для развертывания, Nginx для публикации веб-контура и проксирования запросов.

Для подготовки и проверки окружения используются команды `uv sync --all-packages --group dev`, `./scripts/run-release-gate.sh`, `./scripts/uv-linters.sh` и `./scripts/uv-typecheck.sh`. Эти команды служат для синхронизации зависимостей, запуска релизного контура, проверки линтинга и статического анализа типов.

При ручной проверке применяются встроенные веб-страницы проекта, API-эндпоинты `/health/`, `/ready/`, `/api/`, `/api/docs/` и `/api/v1/`, а также тестовые сценарии из каталогов `tests/`, `src/web/apps/*/tests`, `src/ml/tests` и `src/graph/tests`.

Если в исходной инфраструктуре проекта закреплены точные версии браузеров, СУБД и дополнительных утилит, они указываются в финальной редакции документа отдельным перечнем.

5.3. Порядок проведения испытаний

Испытания выполняются поэтапно. Сначала поднимается и проверяется состав окружения: выполняется синхронизация зависимостей, затем запуск контейнерного стека и проверка готовности сервисов. После этого контролируется доступность базовых точек входа приложения и корректность конфигурации публикации веб-контура.

На следующем этапе проверяются сценарии аутентификации и подтверждения email, после чего выполняются пользовательские сценарии для студента, заказчика и представителя ЦППРП. Для студента проверяются каталог проектов, карточка проекта, избранное, интересы, рекомендации и подача заявки. Для заказчика проверяются создание и редактирование проекта, отправка на модерацию и работа с заявками. Для представителя ЦППРП проверяются очередь модерации, публикация или возврат проекта на доработку, а также операции с дедлайнами и выгрузками.

Затем проверяются интеграционные сценарии импорта проектов из XLSX и через соответствующий API, публикация доменных событий, обновление outbox и синхронизация внешних контуров ML и графа. Если в стенде используется graph-сервис, подтверждается обработка событий и обновление проектной проекции в Neo4j.

После функциональных сценариев выполняются проверки ограничений доступа, обработки некорректных данных, реакций на отсутствующие или просроченные токены, а также контроль того, что ответ сервиса содержит детализированную ошибку без остановки приложения. Завершающим этапом являются повторный запуск ключевых сценариев после очистки тестовых данных и фиксация результатов испытаний.

5.4. Требования к персоналу

Испытания должны проводить лица, знакомые с работой веб-приложений, REST/JSON API и контейнерного развертывания. Испытателю необходимо уметь запускать проект в Docker Compose, пользоваться браузером, просматривать логи и интерпретировать ответы API и статусы HTTP.

Для проверки административных сценариев требуется участник, который может действовать в роли студента, заказчика или сотрудника ЦППРП и корректно воспроизводить соответствующие бизнес-процессы. Для анализа результатов автоматизированных проверок требуется технический исполнитель, способный запускать скрипты релизного контура и сопоставлять фактическое поведение системы с ожидаемым.

Специальная профильная квалификация уровня разработчика не требуется, однако участники испытаний должны понимать назначение сервисов проекта, структуру ролей и границы ответственности между веб-контуром, ML-сервисом, графовым сервисом и хранилищем данных.

6. МЕТОДЫ ИСПЫТАНИЙ

6.1. Проверка требований к технической документации

Проверка требований к технической документации выполняется путем сопоставления состава и содержания комплекта документов с требованиями ГОСТ 19.301-79 и с фактическим состоянием проекта. Контролируется наличие технического задания, пояснительной записки, программы и методики испытаний, текста программы и руководства оператора, а также единообразие наименования проекта, его английского обозначения, кода 05.01, ролей пользователей и названий основных модулей системы.

При проверке документации сопоставляются формулировки о функциях, интерфейсах, интеграциях, ограничениях и составе поставки с данными репозитория, схемами API, контрактами событий и тестами. Отдельно проверяется, что в документации не закреплены неподтвержденные сведения о технологиях и сценариях, отсутствующих в текущем коде, а также не допущены противоречия между разделами разных документов.

Результат проверки считается положительным, если все обязательные документы присутствуют, структура разделов сохранена, терминология согласована, а все упомянутые функции и ограничения могут быть подтверждены по исходному коду, схемам интерфейсов и тестовым сценариям.

6.2. Проверка требований к интерфейсу

Проверка требований к интерфейсу выполняется вручную в веб-браузере на страницах, реализованных на Django templates, с обязательной проверкой ролей `student`, `customer` и `cpprp`. Испытания включают вход в систему, подтверждение адреса электронной почты, переходы между разделами, работу форм, отображение ошибок ввода, сохранение введенных данных при частично неуспешной операции и проверку ограничений доступа к чужим разделам.

Дополнительно проверяется согласованность интерфейса клиентской части с серверными ответами: корректность отображения каталога проектов, карточки проекта, формы создания и редактирования проекта, очереди модерации, списка заявок, личного профиля и административных экранов. Для значимых действий проверяются подтверждающие диалоги, а для невалидных данных - наличие понятных сообщений без аварийного завершения работы страницы.

Критерием успешности является доступность всех предусмотренных для роли экранов в поддерживаемых браузерах, единообразная навигация по приложению, корректная обработка ошибок ввода и отсутствие ситуаций, в которых пользователь получает доступ к функциям, не соответствующим его роли.

6.3. Проверка требований к функциональным характеристикам

Функциональные характеристики проверяются по сквозным сценариям, охватывающим основные пользовательские роли и внутренние интеграции системы. Для каждого сценария задаются исходные данные, последовательность действий, ожидаемый результат и критерий успешности. Проверка выполняется на реальных данных тестового стенда с использованием веб-интерфейса, HTTP-запросов к API и, при необходимости, контрольных записей в журналах и outbox-контуре.

6.3.1. Основной интеграционный сценарий

Исходные данные: учетная запись студента с подтвержденной электронной почтой, опубликованный проект в каталоге, заполненные интересы и доступная страница рекомендаций. Действия: пользователь входит в систему, открывает каталог проектов, применяет фильтры и сортировку, переходит в карточку проекта, добавляет проект в избранное, уточняет интересы, получает рекомендации и подает заявку на выбранный проект. Ожидаемый результат: каталог отображает только доступные проекты, карточка содержит актуальные сведения, проект сохраняется в избранном, рекомендации и список заявок обновляются, а поданная заявка получает статус, видимый в личном кабинете. Критерий успешности: все операции завершаются без ошибок авторизации и валидации, повторная отправка заявки блокируется, а изменения состояния сохраняются после обновления страницы и повторного запроса к API.

6.3.2. Проверка создания, редактирования и модерации проекта

Исходные данные: учетная запись заказчика с правом создания проектов, заполненная форма проекта и учетная запись сотрудника ЦППРП с правом модерации. Действия: заказчик создает карточку проекта, сохраняет ее как черновик, при необходимости редактирует поля, отправляет проект на модерацию; затем модератор просматривает очередь, проверяет содержимое карточки и принимает решение о публикации либо возврате на доработку с комментарием. Ожидаемый результат: статус проекта последовательно изменяется в соответствии с принятым решением, история изменений сохраняется, а комментарий модератора отображается в интерфейсе и доступен заказчику. Критерий успешности: проект не публикуется без прохождения модерации, посторонний пользователь не может изменить карточку, а решение модератора корректно отражается в каталоге и административной панели.

6.3.3. Проверка обработки заявок студентов

Исходные данные: опубликованный проект с открытым набором, учетная запись студента и учетная запись заказчика, закрепленного за проектом. Действия: студент подает заявку, при необходимости отзывается от нее, после чего заказчик просматривает очередь заявок и принимает решение об утверждении либо отклонении с указанием причины. Ожидаемый результат: заявка появляется в списке заявок студента и заказчика, ее статус обновляется в соответствии с решением, а причина отклонения сохраняется и отображается в истории обработки. Критерий успешности: система не допускает некорректный переход статусов, исключает повторную обработку одной и той же заявки в уже закрытом состоянии и корректно обновляет счетчики связанных сущностей.

6.3.4. Проверка каталога, поиска и рекомендаций

Исходные данные: набор опубликованных проектов, содержащий различающиеся курсы, образовательные программы, технологии и типы проектов, а также заполненные пользовательские интересы. Действия: пользователь открывает каталог, применяет фильтры, выполняет семантический поиск через рекомендательный фасад и сравнивает выдачу с карточками из каталога; при отсутствии семантического ответа проверяется переход на резервное ранжирование по ключевым словам. Ожидаемый результат: фильтрация и поиск возвращают согласованный список проектов,

рекомендации опираются на данные интересов и обновленного индекса, а первые результаты отображаются без заметной задержки. Критерий успешности: поиск и рекомендации не возвращают пустой результат при наличии подходящих данных, а время вывода первых результатов соответствует требованиям ТЗ.

6.3.5. Проверка импорта и актуализации данных

Исходные данные: файл XLSX с проектами или таблица, подготовленная для импорта, а также права на запуск операции импорта. Действия: выполняется загрузка файла, проверка очистки и нормализации данных, фиксация результата импорта и последующая проверка обновления каталога, рекомендаций и связанных внешних процессов через outbox API доставки событий. Ожидаемый результат: корректные записи попадают в хранилище, некорректные строки отклоняются с сообщениями об ошибке, после завершения импорта формируется событие обновления данных, а индекс поиска и рекомендации актуализируются. Критерий успешности: импорт завершается без потери корректных данных, новые проекты становятся доступны в каталоге, а обновление производных данных фиксируется в течение установленного интервала.

6.3.6. Проверка домена факультета и сопоставления научных руководителей

Исходные данные: записи о преподавателях, публикациях и курсах, а также проект, для которого требуется проверка сопоставления с профилем сотрудника НИУ ВШЭ. Действия: выполняется синхронизация данных факультета, просмотр карточки персоны, проверка связанных публикаций и курсов, затем проверяется обновление связи между проектом и научным руководителем. Ожидаемый результат: данные домена faculty доступны через API, связи между сущностями сохраняются, а изменения отражаются в проектном контуре и в событиях синхронизации. Критерий успешности: сведения о факультете и связи с проектами отображаются согласованно, без расхождений между карточками, списками и ответами API.

6.3.7. Проверка агрегированных показателей и контуров синхронизации

Исходные данные: рабочий набор проектов, заявок и событий изменения данных, а также учетная запись сотрудника ЦППРП. Действия: открывается административный раздел с агрегированными счетчиками, просматриваются сведения о количестве проектов, заявок и утвержденных участников, затем проверяется доступность уведомлений о дедлайнах и обработка записей outbox с подтверждением чтения. Ожидаемый результат: агрегированные показатели соответствуют текущему состоянию данных, уведомления отображаются в административном интерфейсе, а подтверждение обработки событий не приводит к их дублированию. Критерий успешности: значения счетчиков совпадают с фактическими записями в системе, а каждый сценарий синхронизации завершается без ошибок доставки и повторной обработки.

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Лист регистрации изменений									
Номера листов (страниц)					Всего листов (страниц в докум.)	№ документа	Входящий № сопроводительного докум. и дата	Подп.	Дата
Изм.	Измененных	Замененных	Новых	Аннулированных					